

✓ RNN Encoder Decoder [Arjita Yadav: ACE080BCT015]

```
# requirements
from __future__ import unicode_literals, print_function, division
from io import open
import unicodedata
import re
import random

import torch
import torch.nn as nn
from torch import optim
import torch.nn.functional as F

import numpy as np
from torch.utils.data import TensorDataset, DataLoader, RandomSampler

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

torch.set_default_device(device)
print(f"Using device = {torch.get_default_device()}")
```

Using device = cpu

```
SOS_token = 0 # Start of the Sentence
EOS_token = 1 # End of the Sentence

class Lang:
    def __init__(self, name):
        self.name = name
        self.word2index = {}
        self.word2count = {}
        self.index2word = {0: "SOS", 1: "EOS"}
        self.n_words = 2 # Count SOS and EOS

    def addSentence(self, sentence):
        for word in sentence.split(' '):
            self.addWord(word)

    def addWord(self, word):
        if word not in self.word2index:
            self.word2index[word] = self.n_words
            self.word2count[word] = 1
            self.index2word[self.n_words] = word
            self.n_words += 1
        else:
            self.word2count[word] += 1
```

```
# Turn a Unicode string to plain ASCII, thanks to
def unicodeToAscii(s):
    return ''.join(
        c for c in unicodedata.normalize('NFD', s)
        if unicodedata.category(c) != 'Mn'
    )

# Lowercase, trim, and remove non-letter characters
def normalizeString(s):
    s = unicodeToAscii(s.lower().strip())
    s = re.sub(r"([.!?])", r" \1", s)
    s = re.sub(r"^a-zA-Z!?.+ ", r" ", s)
    return s.strip()
```

```
def readLangs(path:str):
    lang1 = 'eng'; lang2 = 'fra'
    print("Reading lines...")

    # Read the file and split into lines
    lines = open(path, encoding='utf-8').\
        read().strip().split('\n')

    # Split every line into pairs and normalize (english to french)
```

```

pairs = [[normalizeString(s) for s in l.split('\t')] for l in lines]

# Reverse pairs: English-French -> French-English
pairs = [list(reversed(p)) for p in pairs]

# Input is French, output is English
input_lang = Lang(lang2)
output_lang = Lang(lang1)

return input_lang, output_lang, pairs

```

```

MAX_LENGTH = 5

eng_prefixes = (
    "i am ", "i m ",
    "he is", "he s ",
    "she is", "she s ",
    "you are", "you re ",
    "we are", "we re ",
    "they are", "they re "
)

def filterPair(p):
    return len(p[0].split(' ')) < MAX_LENGTH and \
        len(p[1].split(' ')) < MAX_LENGTH and \
        p[1].startswith(eng_prefixes)

def filterPairs(pairs):
    return [pair for pair in pairs if filterPair(pair)]

```

```

def prepareData(path):
    input_lang, output_lang, pairs = readLangs(path)
    print("Read %s sentence pairs" % len(pairs))
    pairs = filterPairs(pairs)
    print("Trimmed to %s sentence pairs" % len(pairs))
    print("Counting words...")
    for pair in pairs:
        input_lang.addSentence(pair[0])
        output_lang.addSentence(pair[1])
    print("Counted words:")
    print(input_lang.name, input_lang.n_words)
    print(output_lang.name, output_lang.n_words)
    return input_lang, output_lang, pairs

```

```

PATH = r'/content/data.txt'

input_lang, output_lang, pairs = prepareData(PATH)
print(random.choice(pairs))

output_lang.word2index['am'] # try different English words.

```

```

Reading lines...
Read 135842 sentence pairs
Trimmed to 3272 sentence pairs
Counting words...
Counted words:
fra 1757
eng 967
['je suis degoute', 'i m disgusted']
15

```

```

class EncoderRNN(nn.Module):
    def __init__(self, input_size, hidden_size, dropout_p=0.1):
        super(EncoderRNN, self).__init__()
        self.hidden_size = hidden_size

        self.embedding = nn.Embedding(input_size, hidden_size)
        self.rnn = nn.RNN(hidden_size, hidden_size, batch_first=True)
        self.dropout = nn.Dropout(dropout_p)

    def forward(self, input):
        embedded = self.dropout(self.embedding(input))
        output, hidden = self.rnn(embedded)
        return output, hidden

```

```

class DecoderRNN(nn.Module):
    def __init__(self, hidden_size, output_size):
        super(DecoderRNN, self).__init__()
        self.embedding = nn.Embedding(output_size, hidden_size)
        self.rnn = nn.RNN(hidden_size, hidden_size, batch_first=True)
        self.out = nn.Linear(hidden_size, output_size)

    def forward(self, encoder_outputs, encoder_hidden, target_tensor=None):
        batch_size = encoder_outputs.size(0)
        decoder_input = torch.empty(batch_size, 1, dtype=torch.long, device=device).fill_(SOS_token)
        decoder_hidden = encoder_hidden
        decoder_outputs = []

        for i in range(MAX_LENGTH):
            decoder_output, decoder_hidden = self.forward_step(decoder_input, decoder_hidden)
            decoder_outputs.append(decoder_output)

            if target_tensor is not None:
                # Teacher forcing: Feed the target as the next input
                decoder_input = target_tensor[:, i].unsqueeze(1) # Teacher forcing
            else:
                # Without teacher forcing: use its own predictions as the next input
                _, topi = decoder_output.topk(1) # values, index of the highest-scoring word
                decoder_input = topi.squeeze(-1).detach() # detach from history as input (removes the last dimension before de

        decoder_outputs = torch.cat(decoder_outputs, dim=1)
        decoder_outputs = F.log_softmax(decoder_outputs, dim=-1)
        return decoder_outputs, decoder_hidden, None # We return `None` for consistency in the training loop

    def forward_step(self, input, hidden):
        output = self.embedding(input)
        output = F.relu(output)
        output, hidden = self.rnn(output, hidden)
        output = self.out(output)
        return output, hidden

```

```

def indexesFromSentence(lang, sentence):
    return [lang.word2index[word] for word in sentence.split(' ')]

def tensorFromSentence(lang, sentence):
    indexes = indexesFromSentence(lang, sentence)
    indexes.append(EOS_token)
    return torch.tensor(indexes, dtype=torch.long, device=device).view(1, -1)

def tensorsFromPair(pair):
    input_tensor = tensorFromSentence(input_lang, pair[0])
    target_tensor = tensorFromSentence(output_lang, pair[1])
    return (input_tensor, target_tensor)

def get_dataloader(batch_size):
    input_lang, output_lang, pairs = prepareData(path=PATH)

    n = len(pairs)
    input_ids = np.zeros((n, MAX_LENGTH), dtype=np.int32)
    target_ids = np.zeros((n, MAX_LENGTH), dtype=np.int32)

    for idx, (inp, tgt) in enumerate(pairs):
        inp_ids = indexesFromSentence(input_lang, inp)
        tgt_ids = indexesFromSentence(output_lang, tgt)
        inp_ids.append(EOS_token)
        tgt_ids.append(EOS_token)
        input_ids[idx, :len(inp_ids)] = inp_ids
        target_ids[idx, :len(tgt_ids)] = tgt_ids

    train_data = TensorDataset(torch.LongTensor(input_ids).to(device),
                               torch.LongTensor(target_ids).to(device))

    train_sampler = RandomSampler(train_data)
    train_dataloader = DataLoader(train_data, sampler=train_sampler, batch_size=batch_size)
    return input_lang, output_lang, train_dataloader

```

Training Loop

```
def train_epoch(dataloader, encoder, decoder, encoder_optimizer,
               decoder_optimizer, criterion):

    total_loss = 0
    for data in dataloader:
        input_tensor, target_tensor = data

        encoder_optimizer.zero_grad()
        decoder_optimizer.zero_grad()

        encoder_outputs, encoder_hidden = encoder(input_tensor)
        decoder_outputs, _, _ = decoder(encoder_outputs, encoder_hidden, target_tensor) # using teacher forcing

        loss = criterion(
            decoder_outputs.view(-1, decoder_outputs.size(-1)),
            target_tensor.view(-1)
        )
        loss.backward()

        encoder_optimizer.step()
        decoder_optimizer.step()

        total_loss += loss.item()

    return total_loss / len(dataloader)
```

```
import time
import math

def asMinutes(s):
    m = math.floor(s / 60)
    s -= m * 60
    return '%dm %ds' % (m, s)

def timeSince(since, percent):
    now = time.time()
    s = now - since
    es = s / (percent)
    rs = es - s
    return '%s (- %s)' % (asMinutes(s), asMinutes(rs))
```

```
import matplotlib.pyplot as plt
plt.switch_backend('agg')
import matplotlib.ticker as ticker
import numpy as np

def showPlot(points):
    plt.figure()
    fig, ax = plt.subplots()
    # this locator puts ticks at regular intervals
    loc = ticker.MultipleLocator(base=0.2)
    ax.yaxis.set_major_locator(loc)
    plt.plot(points)
```

```
def train(train_dataloader, encoder, decoder, n_epochs, learning_rate=0.001,
         print_every=100, plot_every=100):
    start = time.time()
    plot_losses = []
    print_loss_total = 0 # Reset every print_every
    plot_loss_total = 0 # Reset every plot_every

    encoder_optimizer = optim.Adam(encoder.parameters(), lr=learning_rate)
    decoder_optimizer = optim.Adam(decoder.parameters(), lr=learning_rate)
    criterion = nn.NLLLoss()

    for epoch in range(1, n_epochs + 1):
        loss = train_epoch(train_dataloader, encoder, decoder, encoder_optimizer, decoder_optimizer, criterion)
        print_loss_total += loss
        plot_loss_total += loss

        if epoch % print_every == 0:
            print_loss_avg = print_loss_total / print_every
            print_loss_total = 0
            print('%s (%d %d%%) %.4f' % (timeSince(start, epoch / n_epochs),
                                         epoch, epoch / n_epochs * 100, print_loss_avg))
```

```

        if epoch % plot_every == 0:
            plot_loss_avg = plot_loss_total / plot_every
            plot_losses.append(plot_loss_avg)
            plot_loss_total = 0

    showPlot(plot_losses)

```

✓ Evaluation Code

```

def evaluate(encoder, decoder, sentence, input_lang, output_lang):
    with torch.no_grad():
        input_tensor = tensorFromSentence(input_lang, sentence)

        encoder_outputs, encoder_hidden = encoder(input_tensor)
        decoder_outputs, decoder_hidden, decoder_attn = decoder(encoder_outputs, encoder_hidden)

        _, topi = decoder_outputs.topk(1)
        decoded_ids = topi.squeeze()

        decoded_words = []
        for idx in decoded_ids:
            if idx.item() == EOS_token:
                decoded_words.append('<EOS>')
                break
            decoded_words.append(output_lang.index2word[idx.item()])
    return decoded_words, decoder_attn

```

```

def evaluateRandomly(encoder, decoder, n=10):
    for i in range(n):
        pair = random.choice(pairs)
        print('>', pair[0])
        print('=', pair[1])
        output_words, _ = evaluate(encoder, decoder, pair[0], input_lang, output_lang)
        output_sentence = ' '.join(output_words)
        print('<', output_sentence)
        print('')

```

✓ Training and Evaluating

```

hidden_size = 128
batch_size = 32
EPOCHS = 200

input_lang, output_lang, train_dataloader = get_dataloader(batch_size)

encoder = EncoderRNN(input_lang.n_words, hidden_size).to(device)
decoder = DecoderRNN(hidden_size, output_lang.n_words).to(device)

train(train_dataloader, encoder, decoder, EPOCHS, print_every=5, plot_every=5)

```

```

Reading lines...
Read 135842 sentence pairs
Trimmed to 3272 sentence pairs
Counting words...
Counted words:
fra 1757
eng 967
0m 12s (- 8m 15s) (5 2%) 1.9980
0m 24s (- 7m 52s) (10 5%) 1.3081
0m 37s (- 7m 41s) (15 7%) 1.1157
0m 51s (- 7m 39s) (20 10%) 0.9813
1m 3s (- 7m 23s) (25 12%) 0.8730
1m 15s (- 7m 9s) (30 15%) 0.7766
1m 28s (- 6m 55s) (35 17%) 0.6873
1m 40s (- 6m 42s) (40 20%) 0.6019
1m 53s (- 6m 30s) (45 22%) 0.5286
2m 6s (- 6m 18s) (50 25%) 0.4645
2m 18s (- 6m 5s) (55 27%) 0.4111
2m 30s (- 5m 51s) (60 30%) 0.3619
2m 43s (- 5m 38s) (65 32%) 0.3227
2m 55s (- 5m 25s) (70 35%) 0.2854
3m 6s (- 5m 11s) (75 37%) 0.2510
3m 19s (- 4m 58s) (80 40%) 0.2233

```

```

3m 32s (- 4m 47s) (85 42%) 0.2038
3m 45s (- 4m 35s) (90 45%) 0.1794
3m 58s (- 4m 24s) (95 47%) 0.1631
4m 12s (- 4m 12s) (100 50%) 0.1518
4m 25s (- 4m 0s) (105 52%) 0.1371
4m 41s (- 3m 50s) (110 55%) 0.1245
4m 54s (- 3m 37s) (115 57%) 0.1147
5m 7s (- 3m 24s) (120 60%) 0.1106
5m 20s (- 3m 12s) (125 62%) 0.0997
5m 32s (- 2m 59s) (130 65%) 0.0932
5m 45s (- 2m 46s) (135 67%) 0.0893
5m 57s (- 2m 33s) (140 70%) 0.0847
6m 10s (- 2m 20s) (145 72%) 0.0802
6m 23s (- 2m 7s) (150 75%) 0.0810
6m 35s (- 1m 54s) (155 77%) 0.0772
6m 48s (- 1m 42s) (160 80%) 0.0722
7m 0s (- 1m 29s) (165 82%) 0.0721
7m 13s (- 1m 16s) (170 85%) 0.0692
7m 25s (- 1m 3s) (175 87%) 0.0704
7m 38s (- 0m 50s) (180 90%) 0.0695
7m 51s (- 0m 38s) (185 92%) 0.0638
8m 4s (- 0m 25s) (190 95%) 0.0676
8m 17s (- 0m 12s) (195 97%) 0.0618
8m 30s (- 0m 0s) (200 100%) 0.0614

```

```

encoder.eval()
decoder.eval()
evaluateRandomly(encoder, decoder)

```

```

> il est trop saoul
= he s too drunk
< he s too drunk <EOS>

> nous sommes attaquées
= we re being attacked
< i m a salesperson <EOS>

> nous sommes des hommes
= we are men
< we are men <EOS>

> vous êtes très astucieux
= you re very astute
< you re very astute <EOS>

> vous êtes sauf
= you re safe
< he s a senior <EOS>

> je suis vanne
= i m exhausted
< you re all right <EOS>

> tu es trop lent
= you re too slow
< you re too slow <EOS>

> je suis vraiment désolé
= i am truly sorry
< i m very sorry <EOS>

> t es plantée
= you re stuck
< she is my girlfriend <EOS>

> je suis étudiant
= i m a student
< you re all happy <EOS>

```

Discussion

This experiment successfully showed how an RNN Encoder-Decoder model can translate French to English. The model got better at translating short, simple phrases, which we could see from the training loss steadily dropping from 1.9980 to 0.0614 over 200 training cycles, partly thanks to 'teacher forcing.' While it did well with phrases it recognized (like "we are men"), it struggled with less common or new phrases, often giving irrelevant or generic translations (for example, it translated "we're being attacked" as "i'm a salesperson"). These limitations come from using a basic RNN structure, the strict limit of `MAX_LENGTH = 5` for sentence length, the relatively small and

filtered dataset, and not having an attention mechanism. Essentially, the model couldn't really understand context deeply; it mostly relied on surface-level patterns.

Conclusion

In summary, while this basic RNN Encoder-Decoder model can translate simple French-to-English phrases pretty well, it hits a wall when faced with anything complicated or unfamiliar. Its success with familiar phrases highlights the potential of sequence-to-sequence models. However, its struggles with new inputs clearly demonstrate why we need more advanced designs, such as LSTMs/GRUs or attention mechanisms, along with larger datasets and proper ways to measure performance (like BLEU scores), to achieve truly effective neural machine translation. This foundational work gives us valuable insight into the core concepts and challenges involved in building these kinds of systems.

Start coding or [generate](#) with AI.